

UNIVERSIDADE FEDERAL DE SERGIPE

LETICIA DA MATA CAVALCANTI

Atividade 2 (AV2) – Memória, armazenamento, sincronização e desempenho de aplicações de IA generativa: o GenResearch-Servidor

SÃO CRISTÓVÃO - SE

2026



UNIVERSIDADE FEDERAL DE SERGIPE

LETICIA DA MATA CAVALCANTI

Atividade 2 (AV2) – Memória, armazenamento, sincronização e desempenho de aplicações de IA generativa: o GenResearch-Servidor

Atividade apresentada à disciplina de Sistemas Operacionais como requisito para obtenção de nota.

Prof. Dr.: Glauco de Figueiredo Carneiro

SÃO CRISTÓVÃO - SE

2026

1. Tabela de contribuição individual

Integrante	Contribuição	Participação
Leticia da Mata Cavalcanti	Definição do recorte; implementação do serviço e dos experimentos; execução; análise; relatório; slides; vídeo.	100%

2. Introdução e problema

Aplicações de IA generativa e de RAG (*retrieval-augmented generation*) são, para o sistema operacional (SO), cargas que combinam arquivos grandes (corpus, índices, *embeddings*, *checkpoints*), memória virtual e cache de páginas, filas e caches compartilhados por várias threads e escrita concorrente em logs. Este trabalho implementa o **GenResearch-Servidor**, um serviço RAG simplificado, e o usa como carga para observar como o SO administra esses recursos e quais custos surgem.

Pergunta norteadora

Como memória principal, memória virtual, armazenamento, caches, filas compartilhadas e mecanismos de sincronização afetam o desempenho, a estabilidade e a correção de uma aplicação de IA generativa ou de RAG?

Recorte experimental

O foco é o SO, não a qualidade do texto (enunciado, §1). Por isso os *embeddings* e a geração são **simulados**, o que o enunciado admite (Parte B: “embeddings reais ou vetores simulados”; “gerar por modelo local ou simular custo computacional”). A simulação preserva o perfil de recursos (espera de serviço externo, CPU proporcional ao contexto, varredura real do índice) e torna tudo reprodutível sem baixar modelo. O corpus é sintético. O recorte cobre as configurações C1–C6 e a Parte D (opções 1 e 2, log concorrente e deadlock isolado). A ideia de *chunking* por palavras com sobreposição e de cosseno sobre vetores normalizados vem de um serviço RAG anterior da autora, reescrito aqui sem dependência dele.

3. Ambiente

Item	Valor
Hospedeiro / Linux	Windows 11, Docker Desktop 28.3.2 (WSL2); **contêiner** Debian GNU/Linux 13, kernel 6.6.87.2-microsoft-standard-WSL2
CPU	Intel Core i5-10210U, 4 CPUs lógicas (2 núcleos × 2 threads)
RAM / swap / página	5,8 GiB / 1,0 GiB (VM) / 4 KiB; sem GPU

Item	Valor
Linguagem e bibliotecas	Python 3.12, numpy 2.1.3, matplotlib 3.9.2, SQLite; data das execuções: 18/10/2026

Os comandos e saídas completas (`uname`, `lscpu`, `free`, `/proc/meminfo`, `lsblk`, `df`, `swapon`, `vmstat`, `iostat`) estão em `results/env/`. **MemFree** é a RAM totalmente ociosa; **MemAvailable** estima o que pode ser dado a novos processos sem *swap*, incluindo cache de páginas e *slabs* reclamáveis; o **cache de páginas** guarda arquivos já lidos e não é memória desperdiçada (a 2ª leitura evita o disco, §9, e ele é devolvido sob pressão); o **swap** recebe páginas anônimas expulsas da RAM. Para datasets, índices, *embeddings* e *checkpoints*: a 1ª leitura paga E/S, as seguintes não, e um arquivo maior que a memória disponível não cabe todo no cache.

4. Arquitetura

```
2 produtores --put--> BoundedBuffer(empty/full/mutex) --get--> N threads
worker: cache --miss--> embedding --> varredura do índice (read | mmap)
        --> SQLite --> contexto limitado --> geração simulada --> cache
logs (mutex) | métricas: /proc, getrusage, cgroup v2
```

- **Ingestão** (`store.build_index`): lê `.txt`, fragmenta em trechos de 100 palavras (20 de sobreposição), gera vetores float32 de 256 dimensões e grava `vectors.npy` e `meta.sqlite`.
- **Recuperação:** top-k (k = 5) por produto escalar, em blocos de 32.768 linhas, igual para índice na RAM (`np.load`) ou mapeado (`mmap_mode="r"`); contexto limitado em bytes (≈ tokens × 4).
- **Geração simulada:** `sleep` de 20 ms (serviço externo) mais SHA-256 repetido sobre o contexto (CPU).
- **Trabalhadores = threads:** a carga é mista (o `sleep`, o `matmul` do numpy e o `hashlib` liberam o GIL); processos custariam cópia do índice e IPC. A escolha foi testada em §10.1.

5. Dados, corpus, embeddings, índice e cache

O corpus (`corpus.py`, semente fixa) tem 12 tópicos; 70% das palavras de cada documento vêm do vocabulário do tópico e 30% do global, de modo que consultas de um tópico recuperam trechos dele. Nada é real ou pessoal. O **cache compartilhado** é LRU ou FIFO, com chave canônica (palavras únicas ordenadas, sem *stopwords*), protegido por mutex, e registra acertos, erros, substituições e o tempo economizado.

Documentos	Trechos	Corpus (texto)	vectors.npy
500	6.500	≈ 4 MB	6,4 MB
2.000 (C3, C4, C6)	26.000	≈ 17 MB	25 MB
8.000	104.000	≈ 68 MB	102 MB
24.000	312.000	≈ 200 MB	305 MB

6. Processos, threads, filas e recursos compartilhados

Há um processo Python com 2 threads produtoras, N trabalhadoras e a thread principal; o numpy cria ainda um *pool* de threads do OpenBLAS (§10.1). Os recursos compartilhados e sua proteção:

Recurso	Risco sem proteção	Proteção
Fila de requisições	perda, duplicação, estouro	`BoundedBuffer`: semáforos `empty`/`full` + mutex; FIFO verificado
Cache de respostas	atualização perdida, computação duplicada	`Lock` na estrutura e nos contadores
Arquivo de log	linhas intercaladas	`Lock` em `SafeLogger`
Conexão SQLite	uso entre threads	uma conexão somente-leitura por thread
Índice vetorial	–	somente leitura após a ingestão

7. Estratégia de sincronização

Adotei semáforos contadores (`empty`, `full`) mais mutex no buffer limitado (Opção 1); mutex global e lock por chave (*single-flight*) no cache (Opção 2); mutex no log; ordenação global de locks e `acquire` com *timeout* e recuo para o deadlock. O trecho abaixo mostra o produtor do buffer:

```
def put(self, item, timeout=None):
    if not self._empty.acquire(timeout=timeout):  # há vaga? (empty)
        return False                              # cheia: rejeita
    with self._mutex:                             # protege o deque
        self._buf.append((self._seq, now(), item))
        self._seq += 1
    self._full.release()                          # sinaliza item (full)
    return True
```

Os semáforos ficam **fora** da seção crítica: se o produtor pegasse o mutex antes de `empty` com o buffer cheio, ficaria bloqueado segurando o mutex e o consumidor não conseguiria liberar espaço (deadlock).

8. Metodologia

Cada configuração foi repetida **3 vezes**, com o mesmo corpus e a mesma aplicação em cada comparação. Fatores variados: workers (1, 2, 4, 8), capacidade da fila (4, 64), tamanho do corpus (6 a 305 MB de índice), contexto (500, 2.000, 8.000 B), capacidade (0–512) e

política (LRU/FIFO) do cache, `mmap`, padrão de leitura, estado do cache de páginas e limite de memória (cgroup). Métricas: latência (média, mediana, p95), vazão, RSS/VSZ, `MemAvailable`, faltas de página, bytes lidos, fila, utilização, acertos/erros, perdas/duplicações. O estado "frio" usa `posix_fadvise(DONTNEED)` antes de cada execução (frio na VM Linux; o Windows ainda pode cachear o disco virtual). Comandos: `python scripts/run_all.py`; dados brutos em `results/`; tabelas completas no Anexo A.

9. Resultados: memória, armazenamento, paginação e E/S

9.1 C1 × C2 – `read()` convencional × `mmap` (Parte E)

Arquivo aleatório de 16, 128 e 768 MB; leitura sequencial em blocos de 1 MiB (`read` em blocos, `read` do arquivo inteiro, `mmap`) e 3.000 acessos de 4 KiB; execução fria e quente. Valores para 768 MB (média de 3):

Método / padrão	Cache	Leitura (ms)	ΔRSS (MB)	Faltas maiores	Lido do disco (MB)
`read` em blocos, seq.	fria	755	0	0	768
`read` em blocos, seq.	quente	331	0	0	0
`read` tudo, seq.	fria	1.369	768	0	768
`mmap`, seq.	fria	1.064	768	6.146	768
`mmap`, seq.	quente	322	768	0	0
`pread` 4 KiB, aleat.	fria	637	0	0	**23**
`mmap`, aleat.	fria	1.410	165	2.441	**287**

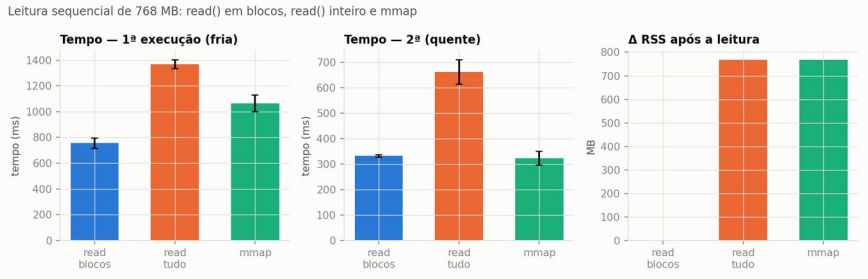


Figura 1 – Leitura sequencial de 768 MB (média ± desvio-padrão): tempo frio, quente e ΔRSS.

- A 2ª execução foi 2,1–3,3× mais rápida e não leu nada do disco: é o **cache de páginas**.
- O `mmap` abre em ~20–40 µs (mapeia, não lê) e o VSZ sobe 768 MB na abertura, mas o RSS só cresce ao tocar as páginas (**VSZ ≠ memória física**); `read()` não gera faltas maiores, o `mmap` gerou 6.146 (≈ 125 KiB cada, leitura antecipada).

- `read` do arquivo inteiro é o pior caso (2× mais lento, +768 MB de RSS): é o que ocorre ao carregar um *checkpoint* inteiro. No acesso aleatório frio o `mmap` leu **287 MB** para ~12 MB úteis (12×), contra 23 MB do `pread`, e foi 2,2× mais lento.

9.2 C5 – pressão de memória

(a) Corpus e contexto (1 trabalhador, processo novo por configuração, 1ª leitura fria; dados em `results/c5_scale.csv`): a recuperação cresce **linearmente** com o índice (0,8 ms a 6 MB; ≈ 21 ms a 305 MB; ≈ 70 µs/MB, varredura O(N)) e o RSS o acompanha (44 → 344 MB). `read` e `mmap` têm RSS e latência iguais em regime estacionário; muda *quando* o custo é pago (`read` na abertura; `mmap` em 2.474 faltas maiores na 1ª consulta, a 305 MB). A geração cresce com o contexto (≈ 20 ms com 500 B; ≈ 100–120 ms com 8.000 B) e, com limite de 500 B, quase nenhum trecho (~700 B) cabe: o limite de contexto trunca informação.

(b) Índice maior que o limite de memória (cgroup, swap do contêiner desligado): índice sintético de 1.024 MB, 15 consultas de varredura completa, cache de páginas despejado antes de cada execução.

Limite	Modo	Resultado (3 de 3 repetições)	Mediana/ consulta	Faltas maiores	Lido do disco
2.048 MB	`read`	conclui 15/15	70–103 ms	0	1,0 GB
2.048 MB	`mmap`	conclui 15/15	89–153 ms	≈ 8.300	1,0 GB
384 MB	**`read`**	**morto pelo OOM killer (exit 137)**	–	–	–
384 MB	**`mmap`**	**conclui 15/15, RSS ≈ 399 MB**	**1,2–1,6 s**	**≈ 125.000**	**≈ 15,3 GB**

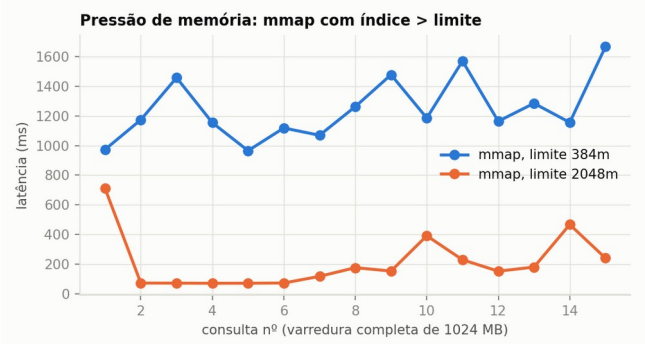


Figura 2 – Latência por consulta com `mmap` e índice de 1 GB: limite de 2 GB × 384 MB.

Com 384 MB, `read` precisa de ~1 GB de memória **anônima** (irreclamável sem swap) e o kernel mata o processo. O `mmap` usa páginas de **arquivo** (limpas, reclamáveis): o kernel as descarta e relê, então o processo sobrevive, mas com *thrashing*: cada varredura de 1 GB excede o cache (≈ 384 MB), que nunca retém o que a próxima varredura precisa, e todas as 15 relêem do disco (≈ 15 × 1.024 MB = 15,3 GB), com consultas 13–17× mais lentas. **O**

swap não foi exercitado (`--memory-swap` igual a `--memory`); a paginação observada é de arquivo, não de swap.

10. Resultados: concorrência, cache e correção

10.1 C3 – concorrência e fila

400 consultas distintas, cache desligado, 2 produtores, 2.000 documentos (média ± desvio de 3 repetições):

Fila	Workers	Vazão (req/s)	p95 do serviço (ms)	Espera na fila (ms)	Fila média / máx
4	1 / 2 / 4 / 8	25,3 / 36,5 / 43,1 / 46,0	51 / 73 / 138 / 274	157 / 106 / 87 / 76	≈ 3,0–3,5 / 4
64	1 / 2 / 4 / 8	25,4 / 39,4 / 47,0 / 43,6 (±6,0)	51 / 67 / 126 / 292	2.341 / 1.483 / 1.248 / 1.340	≈ 58 / 64

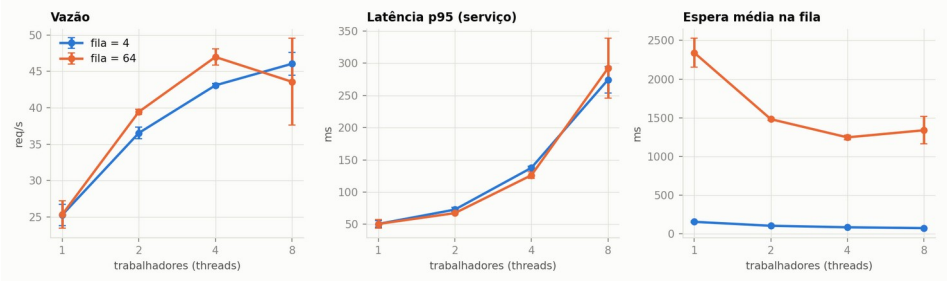


Figura 3 – Vazão, p95 do serviço e espera na fila × número de trabalhadores (filas de 4 e 64).

- **A concorrência aumentou a vazão, com retorno decrescente:** 4 workers = 1,7–1,85× a de 1; com 8, sem ganho (47,0 → 43,6 req/s) e p95 ~5× pior: com 4 CPUs lógicas (2 núcleos físicos) e parte do trabalho sob o GIL, mais threads só trazem disputa e trocas de contexto.
- **Fila maior não aumenta vazão, aumenta latência:** a fila ficou quase sempre cheia (58 de 64); espera de 1,2–2,3 s (fila 64) contra ~0,1 s (fila 4), como prevê a lei de Little. O limite dá *backpressure*. **Correção:** 0 perdidas, 0 duplicadas e 0 violações de FIFO.
- **Hipótese testada:** em `ps -L` havia 3 threads a ~82% de CPU quase sem trocas voluntárias (espera ativa do OpenBLAS?). Com `OPENBLAS_NUM_THREADS=1` a vazão subiu ≈ 10–15% com 1–2 workers, mas a saturação acima de 2 persistiu: a causa total não foi isolada.

10.2 C4 – cache compartilhado

400 requisições, 2 workers (cenários: consultas distintas, 50%/90% de repetição exata e 50% de paráfrases):

Cenário (LRU)	Capacidade	Acerto	Erros	Tempo economizado	Tempo total
distintas	512	0%	400	0 s	11,3 ± 1,0 s
repetidas 50%	512	44,7%	221	9,3 s	5,7 s (sem cache: 12,0 s)
repetidas 90%	16 / 64	74,0% / 89,2%	104 / 43	16,2 / 19,2 s	2,7 / 1,25 s (sem cache: 9,9 s)
paráfrase 50%	512	50,9%	196	10,9 s	5,2 s

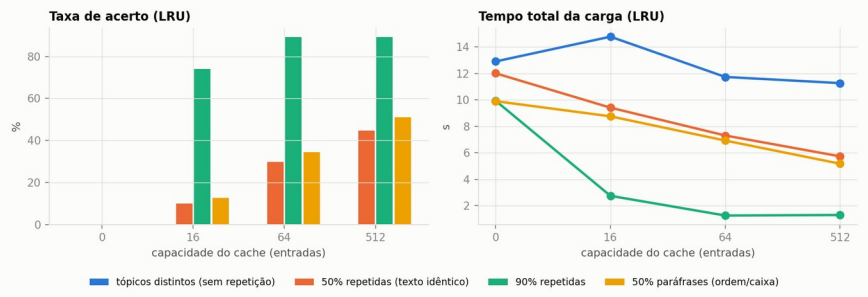


Figura 4 – Taxa de acerto e tempo total × capacidade do cache, por cenário (LRU).

O cache reduziu tempo e operações de recuperação **quando há repetição** (89,2% de acerto com 90% de repetição: 9,9 s → 1,25 s, ≈ 8×); com consultas distintas o acerto é 0% e só há custo. LRU e FIFO empataram (74,0% × 73,7%): as repetições foram uniformes. Com o cache desligado o mesmo trabalho levou de 9,9 a 12,9 s (± 2,7 s), então diferenças < ~25% não são significativas aqui.

10.3 C6 – falhas controladas

Cenário	Resultado (média de 3)
Arquivo/índice ausente	40 requisições → 40 erros controlados, 80 retentativas; o processo não caiu
Índice indisponível por 0,6 s	120/120 concluídas com sucesso após retentativas com recuo (2,2 s)
Fila saturada, produtor com *timeout*	8 concluídas, **112 rejeitadas** (93%) e registradas; 0 perdas silenciosas
Fila saturada, produtor bloqueia	120/120 concluídas em 4,0 s (*backpressure*)
Serviço externo falhando 30%	sem retentativa: 41 erros (34%); com até 3: 1 erro, 55 retentativas, +88% de tempo
Limite de memória	§9.2(b): `read` morto por OOM (exit 137); `mmap` sobrevive com *thrashing*

10.4 Parte D – sincronização, falhas demonstradas e correções

As janelas de corrida foram ampliadas **de propósito** (`sleep(0)` entre leitura e escrita e `sys.setswitchinterval(1e-6)`): injeção de falha controlada; no código "natural" a corrida seria bem mais rara.

D1 – cache (8 threads × 1.500 ops)	Atualizações perdidas	Computações (ideal 40)	Tempo
sem proteção	**10.334 de 12.000 (86%)**	62 (**22 duplicadas**)	0,36 s
mutex global	0	40	0,68 s
lock por chave (*single-flight*)	0	40	0,56 s

- **D1:** `hits += 1` (ler–somar–escrever) perde atualizações e duas threads calculam a mesma chave; o lock por chave corrige e é ~18% mais rápido que o global (chaves distintas computam em paralelo).
- **D2 – log:** sem lock, **1.817 de 2.400 linhas (76%)** saíram corrompidas; com mutex, 0.
- **D3 – produtor/consumidor** (capacidades 1, 4, 64; 18 execuções): 0 perdidos, 0 duplicados, 0 violações de FIFO; com consumidor lento o produtor bloqueia ≈ 1,2 s; com produtor lento os consumidores esperam ≈ 1,0 s.
- **D4 – deadlock** (A→B / B→A, em subprocesso): 3/3 não terminaram; após 3 s o pai o classificou como deadlock (threads em `futex_wait_queue`) e o encerrou (`kill`). Correções: **ordenação global de locks** (0,30 s) e **`acquire` com timeout + recuo** (0,60 s). *Starvation* não foi demonstrada.

Evidências de SO (`results/evidence/`: `ps -L`, `/proc/<pid>/status`, `pidstat -t`, `vmstat`, `iostat`, `strace -c -f`): com 4 trabalhadores havia 11 threads; **`futex` respondeu por 70% do tempo em syscalls** (3.919 chamadas), seguido de `clock_nanosleep` (16%). Para o mesmo arquivo de 16 MB, `read` aparece como sequência de `read(...)` e `mmap` como um único `mmap(NULL, 16777216, PROT_READ, MAP_SHARED, ...)`: a leitura passa a ocorrer por faltas de página.

11. Relação com Sistemas Operacionais e IA generativa

- **Memória virtual e cache de páginas:** carregar *embeddings*/*checkpoints* com `read()` exige memória anônima do tamanho do arquivo (OOM se não couber); `mmap` evita a cópia e funciona acima da RAM, mas vira *thrashing* se o conjunto de trabalho não cabe no cache. A 1ª carga é fria; as seguintes, quentes.

- **RAG:** a busca por força bruta é O(N); índices aproximados (HNSW, IVF) reduzem esse custo e o limite de contexto é limite de custo e de qualidade. Fila limitada dá *backpressure* e cache de resposta só paga com repetição.
- **Concorrência:** a vazão satura por GIL/CPU; em inferência real a contenção seria por GPU e memória do dispositivo.

12. Limitações

- Contêiner sobre WSL2 sobre Windows: “frio” é frio no cache de páginas da VM; 2 núcleos físicos e ruído do hospedeiro (9,9–12,9 s para trabalho equivalente no C4).
- Embeddings e geração simulados: sem GPU, VRAM, KV-cache ou pesos; **não generaliza** para modelos maiores, múltiplas GPUs ou ambientes distribuídos (dominam a transferência host ↔ dispositivo, a memória da GPU e a rede).
- Corpus pequeno (≤ 305 MB de índice); taxas de corrida (86%, 76%) valem para janelas ampliadas; swap e starvation não testados; causa total da saturação com > 2 workers não isolada; 3 repetições.

13. Conclusão

O serviço mostrou, com evidências reprodutíveis, que: (i) o cache de páginas explica a diferença entre a 1ª e a 2ª leitura, e carregar tudo com `read()` é o caminho mais caro em memória; (ii) `mmap` troca RAM anônima por paginação sob demanda: permite operar um índice de 1 GB com 384 MB, onde `read` é morto por OOM, ao custo de *thrashing* (~15× mais lento, 15 GB lidos); (iii) mais threads não significam mais vazão (saturação em 2–4 workers, piora do p95 com 8) e filas grandes viram latência; (iv) o cache só compensa com repetição, mas então reduz o tempo em ≈ 8×; (v) sem exclusão mútua o cache e o log ficam incorretos (86% de atualizações perdidas, 76% de linhas corrompidas) e a ordem de aquisição de locks pode causar deadlock, evitável por ordenação global ou *timeouts*.

14. Referências

[1] MAZIERO, Carlos A. *Sistemas Operacionais: Conceitos e Mecanismos*. Curitiba: DInf-UFPR, 2019. Caps. 2, 4, 5 e 6, e slides sobre comunicação e coordenação entre tarefas.

[2] SILBERSCHATZ, A.; GALVIN, P.; GAGNE, G. *Sistemas Operacionais: Conceitos e Aplicações*. Campus, 2001.

[3] TANENBAUM, A. *Sistemas Operacionais Modernos*. 2. ed. Pearson, 2003.

[4] LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020. Disponível em: https://arxiv.org/abs/2005.11401.

[5] LINUX. *man 2 mmap*, *man 2 posix_fadvise*, *man 5 proc*, *man 2 getrusage*, *man 7 cgroups* e documentação do cgroup v2 (`memory.max`, `memory.events`, `memory.stat`).

[6] PYTHON SOFTWARE FOUNDATION. Documentação de `threading`, `mmap` e `sqlite3`; NUMPY. `numpy.load` com `mmap_mode`.

[7] CARNEIRO, G. F. *Sistemas Operacionais como Infraestrutura para IA Generativa* (slides da aula de 26/08/2026). UFS, 2026.

15. Materiais

GitHub: https://github.com/leticiacavalcaanti/SO_UFS_2026_2_AV2_Cavalcanti_Leticia

Link do vídeo: https://drive.google.com/file/d/1ZTenxEhflYIi4mIGWAJvMkZqFxuyeKsG/view?usp=sharing

Commit da entrega: `6a4bcad` (experimentos); evidências, logs e dados brutos em `results/`; instruções de reprodução no `README.md` (`python scripts/run_all.py`).

16. Participação nas tarefas

Leticia Cavalcanti – trabalho individual: definição do recorte, desenvolvimento do código-fonte do serviço e dos experimentos, execução e coleta das evidências, análise dos resultados, elaboração do relatório, dos slides e do vídeo.

17. Apoio na elaboração do conteúdo escrito

Durante o desenvolvimento deste trabalho foi utilizado o Claude Code (Anthropic, modelo Claude Sonnet 5.5) como ferramenta de apoio para:

- Estruturar o repositório e redigir o código do serviço e dos scripts de experimento, executados e conferidos no ambiente da discente (os números do relatório vêm de `results/`).
- Apoiar a análise dos resultados, a redação do relatório e dos slides, e a explicação de conceitos de SO.
- Corrigir problemas metodológicos identificados durante a revisão dos resultados (cache de páginas no C5, RSS contaminado, contagem de retentativas); o detalhamento está em `declaracao_uso_ia.md`.

Anexo A – Tabelas completas dos experimentos

Geradas por `scripts/analyze.py` a partir dos dados brutos (`results/*.csv`). Médias de 3 repetições.

C1/C2 — leitura convencional x mmap (médias de 3 repetições)

MB	método	padrão	cache	abertura (ms)	leitura (ms)	ΔRSS (MB)	ΔVSZ (MB)	falhas maiores	falhas menores	lido do disco (MB)
16	mmap	rand	cold	0.025	102.9	16	16	268	133	16
16	mmap	rand	warm	0.026	6.5	16	16	0	256	0
16	mmap	seq	cold	0.021	29.9	16	16	129	203	16
16	mmap	seq	warm	0.020	7.5	16	16	0	256	0
16	read_all	seq	cold	0.005	27.3	16	16	0	4001	16
16	read_all	seq	warm	0.009	11.3	5	5	0	1365	0
16	read_stream	rand	cold	0.004	328.7	0	0	0	0	15
16	read_stream	rand	warm	0.005	8.6	0	0	0	0	0
16	read_stream	seq	cold	0.005	21.1	0	0	0	175	16
16	read_stream	seq	warm	0.006	8.8	0	0	0	160	0
128	mmap	rand	cold	0.038	619.2	94	128	1205	612	116
128	mmap	rand	warm	0.033	10.7	95	128	0	1618	0
128	mmap	seq	cold	0.026	298.3	128	128	1025	1024	128
128	mmap	seq	warm	0.047	63.1	128	128	0	2048	0
128	read_all	seq	cold	0.005	243.5	128	128	0	32769	128
128	read_all	seq	warm	0.013	130.6	128	128	0	32769	0
128	read_stream	rand	cold	0.008	660.1	0	0	0	0	23
128	read_stream	rand	warm	0.007	10.2	0	0	0	0	0
128	read_stream	seq	cold	0.005	153.1	0	0	0	0	128
128	read_stream	seq	warm	0.006	55.5	0	0	0	0	0
768	mmap	rand	cold	0.036	1409.5	165	768	2441	421	287
768	mmap	rand	warm	0.029	17.6	165	768	0	2812	0
768	mmap	seq	cold	0.021	1063.8	768	768	6146	6144	768
768	mmap	seq	warm	0.024	322.4	768	768	0	12288	0
768	read_all	seq	cold	0.005	1368.7	768	768	0	196609	768
768	read_all	seq	warm	0.012	660.9	768	768	0	196609	0
768	read_stream	rand	cold	0.005	636.7	0	0	0	0	23
768	read_stream	rand	warm	0.007	12.8	0	0	0	0	0
768	read_stream	seq	cold	0.005	754.7	0	0	0	0	768
768	read_stream	seq	warm	0.005	331.4	0	0	0	0	0

C3 — concorrência e fila (400 requisições distintas, cache desligado, 2 produtores)

fila	workers	vazão (req/s)	latência mediana (ms)	p95 (ms)	fila média	fila máx	espera na fila média (ms)	utilização	produtor bloqueado (s)	RSS máx (MB)	threads máx	perdidas	duplicadas
4	1	25.3 ± 1.5	37.4	50.5	3.5	4	157.3	1.00	31.14	114	8	0	0
4	2	36.5 ± 0.8	53.3	72.8	3.3	4	106.2	1.00	21.43	178	9	0	0
4	4	43.1 ± 0.2	89.7	137.6	3.2	4	86.8	0.99	17.91	229	11	0	0
4	8	46.0 ± 1.5	165.9	274.3	3.0	4	76.0	0.99	16.77	301	15	0	0
64	1	25.4 ± 1.9	37.3	50.5	58.5	64	2340.9	1.00	24.17	329	8	0	0
64	2	39.4 ± 0.3	49.5	67.3	58.3	64	1482.6	1.00	15.12	338	9	0	0
64	4	47.0 ± 1.1	82.5	126.1	58.2	64	1247.8	1.00	12.78	357	11	0	0
64	8	43.6 ± 6.0	177.0	292.1	58.0	64	1339.7	0.99	13.41	388	15	0	0

C4 — cache compartilhado (400 requisições, 2 workers)

cenário	capacidade	política	acerto (%)	erros (misses)	substituições	tempo economizado (s)	tempo total (s)	vazão (req/s)	p95 (ms)
distintas	0	lru	0.0	400	0	0.00	12.91 ± 2.73	32.0	100.7
distintas	16	fifo	0.0	400	384	0.00	19.23 ± 4.61	21.8	152.0
distintas	16	lru	0.0	400	384	0.00	14.79 ± 5.04	28.9	119.8
distintas	64	lru	0.0	400	336	0.00	11.74 ± 1.29	34.3	81.3
distintas	512	lru	0.0	400	0	0.00	11.27 ± 1.04	35.7	79.3
parafrase50	0	lru	0.0	400	0	0.00	9.91 ± 0.24	40.4	66.2
parafrase50	16	fifo	13.2	347	326	2.81	9.08 ± 0.07	44.1	69.5
parafrase50	16	lru	12.6	350	330	2.50	8.76 ± 0.20	45.7	67.5
parafrase50	64	lru	34.4	262	194	7.48	6.93 ± 0.11	57.8	67.3
parafrase50	512	lru	50.9	196	0	10.87	5.17 ± 0.17	77.4	66.4
repetidas50	0	lru	0.0	400	0	0.00	12.02 ± 0.76	33.4	79.8
repetidas50	16	fifo	10.2	359	340	2.11	9.20 ± 0.20	43.5	67.7
repetidas50	16	lru	9.9	360	339	2.01	9.42 ± 0.35	42.5	69.8
repetidas50	64	lru	29.7	281	215	6.33	7.31 ± 0.31	54.8	69.0
repetidas50	512	lru	44.7	221	0	9.32	5.73 ± 0.05	69.8	65.8
repetidas90	0	lru	0.0	400	0	0.00	9.94 ± 0.20	40.2	64.6
repetidas90	16	fifo	73.7	105	75	16.60	2.75 ± 0.46	147.8	61.9
repetidas90	16	lru	74.0	104	70	16.19	2.74 ± 0.20	146.3	59.4
repetidas90	64	lru	89.2	43	0	19.20	1.25 ± 0.14	321.4	54.9
repetidas90	512	lru	89.2	43	0	20.56	1.29 ± 0.13	311.7	56.2

C5a — crescimento do corpus e do contexto (1 worker, 150 consultas)

docs	índice (MB)	modo	contexto máx (B)	recuperação mediana (ms)	geração mediana (ms)	p95 total (ms)	RSS máx (MB)	VSZ máx (MB)	falhas maiores	falhas menores	lido do disco (MB)	contexto médio (B)
500	6	read	500	0.82	20.3	24.6	44	448	0	1138	6	22
500	6	read	2000	0.82	34.4	45.9	44	448	0	1155	6	1674
500	6	read	8000	1.06	98.3	133.3	44	449	0	1242	6	7581
500	6	mmap	500	0.76	20.3	24.2	44	448	57	811	6	22
500	6	mmap	2000	0.84	34.2	44.7	44	448	56	833	6	1674
500	6	mmap	8000	1.09	96.2	127.5	44	449	56	915	6	7581
2000	25	read	500	1.98	20.3	26.3	63	467	0	1145	25	43
2000	25	read	2000	2.06	34.3	47.2	63	467	0	1334	25	1655
2000	25	read	8000	2.60	112.7	155.4	64	468	0	1602	25	7617
2000	25	mmap	500	1.96	20.3	26.8	63	467	209	1186	25	43
2000	25	mmap	2000	2.11	37.2	51.4	63	467	208	1211	25	1655
2000	25	mmap	8000	2.66	114.3	149.4	64	468	209	1308	25	7617
8000	102	read	500	9.55	20.4	40.0	140	545	0	23512	102	38
8000	102	read	2000	10.26	43.7	109.4	140	545	0	23560	102	1666
8000	102	read	8000	8.35	112.6	167.4	141	545	0	23631	102	7604
8000	102	mmap	500	10.59	20.5	57.9	140	545	831	23979	102	38
8000	102	mmap	2000	9.08	41.5	71.6	140	545	829	24015	102	1666
8000	102	mmap	8000	8.21	105.7	141.0	140	545	830	24099	102	7604
24000	305	read	500	21.79	20.4	51.9	344	748	0	1247	305	53
24000	305	read	2000	21.55	38.4	79.8	344	748	0	1265	305	1640
24000	305	read	8000	21.48	103.5	160.2	344	748	0	1350	305	7571
24000	305	mmap	500	20.33	20.4	48.1	344	748	2474	3519	305	53
24000	305	mmap	2000	20.95	36.9	75.5	344	748	2474	3545	305	1640
24000	305	mmap	8000	21.87	105.9	173.1	344	748	2475	3640	305	7571

C5b — índice de 1024 MB sob limite de memória do cgroup (swap do contêiner desligado)

limite	modo	rep	exit	OOM kill	abertura (s)	consultas concluídas	mediana consulta (ms)	RSS máx (MB)	falhas maiores	lido do disco (MB)	swap usado (MB)
2048m	read	1	0	não	1.74	15	71	1061	0	1024	1
2048m	read	2	0	não	0.91	15	70	1060	0	1024	1
2048m	read	3	0	não	1.35	15	103	1061	0	1024	1
2048m	mmap	1	0	não	0.00	15	153	1062	8296	1024	1
2048m	mmap	2	0	não	0.00	15	89	1060	8302	1024	1
2048m	mmap	3	0	não	0.00	15	91	1061	8301	1024	1
384m	read	1	137	SIM	-	0	-	-	-	-	-
384m	read	2	137	SIM	-	0	-	-	-	-	-
384m	read	3	137	SIM	-	0	-	-	-	-	-
384m	mmap	1	0	não	0.00	15	1174	398	125343	15339	1

limite	modo	rep	exit	OOM kill	abertura (s)	consultas concluídas	mediana consulta (ms)	RSS máx (MB)	falhas maiores	lido do disco (MB)	swap usado (MB)
384m	mmap	2	0	não	0.00	15	1204	399	124972	15363	1
384m	mmap	3	0	não	0.00	15	1589	399	124790	15363	1

C6 — falhas controladas (média de 3 repetições)

cenário	requisições	ok	erros	rejeitadas	retentativas	perdidas (tot.)	tempo (s)
arquivo_ausente	40	0	40	0	80	0	0.63
indice_indisponivel_recupera	120	120	0	0	10	0	2.19
fila_saturada_rejeita	120	8	0	112	0	0	0.26
fila_saturada_bloqueia	120	120	0	0	0	0	4.01
servico_externo_falha30_retries0	120	79	41	0	0	0	0.96
servico_externo_falha30_retries3	120	119	1	0	55	0	1.81

D1 — cache compartilhado: corrida e correções (8 threads × 1500 ops, 40 chaves)

modo	ops	hits contados	misses contados	atualizações perdidas	computações	computações duplicadas	tempo (s)
unsafe	12000	1614	52	10334	62	22	0.36
global_lock	12000	11960	40	0	40	0	0.68
per_key	12000	11960	40	0	40	0	0.56

D2 — log concorrente

logger	eventos	linhas no arquivo	linhas corrompidas (média)
UnsafeLogger	2400	2400	1817
SafeLogger	2400	2400	0

D3 — produtores/consumidores com buffer limitado (3 produtores × 500 itens, 2 consumidores)

cenário	capacidade	produzidos	consumidos	perdidos	duplicados	violações FIFO	ocupação máx	produtor bloqueado (s)	consumidor esperou (s)	tempo (s)
consumidor_lento	1	1500	1500	0	0	0	1	1.20	0.05	0.41

cenário	capacidade	produzidos	consumidos	perdidos	duplicados	violações FIFO	ocupação máx	produtor bloqueado (s)	consumidor esperou (s)	tempo (s)
consumidor_lento	4	1500	1500	0	0	0	4	1.21	0.05	0.41
consumidor_lento	64	1500	1500	0	0	0	64	1.14	0.05	0.41
produtor_lento	1	1500	1500	0	0	0	1	0.04	1.00	0.50
produtor_lento	4	1500	1500	0	0	0	3	0.01	0.98	0.50
produtor_lento	64	1500	1500	0	0	0	3	0.01	1.00	0.50

D4 — deadlock isolado em subprocesso

modo	concluiu	tempo (s)	resultado
deadlock	NÃO	3.01	DEADLOCK detectado por timeout; processo encerrado (kill)
ordered	sim	0.30	COMPLETED T1,T2
timeout	sim	0.60	COMPLETED T1,T2